ENUNCIADO DE LA PRUEBA DE DESEMPEÑO – MÓDULO 6

Arquitectura Hexagonal + JPA Avanzado + Seguridad JWT + Observabilidad + Microservicios + Docker + Pruebas

---

Caso de uso principal

HelpFlow, una empresa con equipos distribuidos en varias sedes, gestiona las solicitudes de soporte tecnológico mediante correos, mensajes y hojas de cálculo. Esto ha generado problemas significativos:

- Tickets duplicados o extraviados.
- Asignaciones manuales y sin criterios claros.
- Falta de trazabilidad sobre cambios de estado y responsables.
- Incumplimiento de tiempos de atención y resolución.
- Ausencia de autenticación segura y control de accesos.
- Falta de estandarización en errores y validaciones.
- Dificultad para operar en entornos distribuidos.
- Capacidad limitada de pruebas, métricas y monitoreo.

La dirección de HelpFlow ha decidido construir un Sistema Integral de Gestión de Soporte TI, modular, seguro, escalable y con arquitectura profesional de tipo Hexagonal, que permita registrar, priorizar, asignar, escalar y cerrar tickets en entornos reales.

Objetivo de la Prueba

Construir un sistema profesional compuesto por dos microservicios, cumpliendo estándares empresariales de arquitectura, seguridad, pruebas y despliegue. El servicio principal será:

1. support-ticket-service (Servicio central de Tickets de Soporte)

Con arquitectura hexagonal, dominio puro, casos de uso independientes, puertos de entrada y salida, adaptadores REST/JPA, validaciones, seguridad, métricas, pruebas y contenedorización.

2. technician-dispatch-service (Servicio externo simulado de disponibilidad técnica)

Microservicio que devuelve un técnico disponible y un tiempo estimado de atención según la categoría, prioridad y sede del ticket.

El desarrollador deberá integrar TODAS las competencias de las semanas 2–6.

Requerimientos funcionales

1) Gestión de Usuarios y Técnicos

- documento, nombre, correo corporativo, sede, área y estado (ACTIVO / INACTIVO)
    - documento, nombre, salario, fecha de afiliación, estado (*`ACTIVO`* / *`INACTIVO`*)
- Editar información básica y perfil de soporte.
- documento y correo únicos,
    - correo corporativo válido,
    - usuario ACTIVO para registrar tickets.
    - afiliado ACTIVO para solicitar crédito.

---

2) Gestión de Tickets de Soporte TI

Cada ticket debe incluir:

- usuario solicitante
- título y descripción
- categoría (HARDWARE, SOFTWARE, RED, ACCESOS, OTRO)
- prioridad (BAJA, MEDIA, ALTA, CRÍTICA)
- sede y activo afectado
- fecha de creación y fecha límite de SLA
- estado (ABIERTO, ASIGNADO, EN_PROGRESO, RESUELTO, CERRADO, CANCELADO)

Flujo obligatorio:

1. El usuario registra un ticket en estado ABIERTO.
2. El sistema calcula la fecha límite de atención según la prioridad y consulta technician-dispatch-service mediante un adaptador REST.
3. El servicio devuelve el técnico sugerido, su disponibilidad y el tiempo estimado de atención.
4. El sistema aplica reglas internas de asignación:
    - compatibilidad entre categoría y especialidad
    - carga máxima de tickets activos por técnico

- prioridad, sede y cumplimiento del SLA

5. El caso de uso Gestionar Ticket:
    - genera una asignación y registra el historial
    - permite transiciones de estado válidas con observaciones
    - escala automáticamente los tickets próximos a incumplir el SLA
6. La asignación inicial y el registro del historial deben ejecutarse dentro de una transacción.

---

3) Microservicio technician-dispatch-service

Este servicio debe exponer un endpoint:

POST /technician-dispatch

Body:

```
{
  "ticketId": 125,
  "categoria": "RED",
  "prioridad": "ALTA"
}
```

Reglas de respuesta:

La respuesta debe ser determinista para la combinación de ticket, categoría y prioridad, y simular la disponibilidad de técnicos.

➡️ La misma combinación debe devolver el mismo técnico sugerido.
➡️ Combinaciones distintas pueden producir técnicos y tiempos diferentes.

Implementación sugerida:

1. Convertir ticketId + categoría + prioridad en un seed numérico.
2. Seleccionar de forma determinista un técnico simulado y un tiempo estimado.
3. Clasificar el tiempo estimado:
    - 0–2 horas → DISPONIBILIDAD ALTA
    - 3–8 horas → DISPONIBILIDAD MEDIA
    - Más de 8 horas → DISPONIBILIDAD BAJA

Respuesta:

```
{
  "ticketId": 125,
  "tecnicoId": "TEC-014",
  "disponibilidad": "MEDIA",
  "tiempoEstimadoHoras": 4
```



```
}
```

Este microservicio NO usa JPA ni seguridad. Es liviano.

---

4) Seguridad, Roles y Autenticación

Debe implementarse seguridad completa:

- Autenticación con **JWT** (stateless).
- Encriptación de contraseñas con **PasswordEncoder**.
- Roles:
    - ROLE_USUARIO
    - ROLE_TECNICO
    - ROLE_ADMIN
- Endpoints:
    - `/auth/register`
    - `/auth/login`
- Control de acceso:
    - Usuario → crear y consultar únicamente sus tickets
    - Técnico → consultar y gestionar los tickets asignados
    - Admin → acceso completo, reasignación y configuración de SLA

---

5) Validaciones, Errores Estándar y Manejo Global

Obligatorio:

- Validación avanzada con Bean Validation:
    - Validaciones cruzadas (transición de estado, prioridad, técnico habilitado y fecha límite de SLA)
- Manejo global con `@ControllerAdvice`
- Formato de error **ProblemDetail (RFC 7807)**:
    - type
    - title
    - status
    - detail

- instance
- **timestamp**
- **traceId**

- Logging estructurado
- Personalización de errores JPA / acceso denegado / validaciones

---

6) Persistencia, Lazy/Eager y Transacciones

Implementar:

- JPA + Hibernate avanzado
- Relaciones:
  - Usuario 1–N Tickets
  - Ticket 1–N Historiales y N–1 Técnico asignado
- Evitar N+1 con `@EntityGraph`, `join fetch` o `batch-size`
- Asignar o escalar ticket → proceso completo dentro de un @Transactional
- Flyway con:
  - V1__schema
  - V2__asignaciones_historial_sla
  - V3__datos_iniciales (opcional)

---

7) Pruebas Unitarias, Integración y Testcontainers

Obligatorio:

Unitarias (JUnit + Mockito)

- Casos de uso puros del dominio de soporte
- Mock del TechnicianDispatchPort

Integración (Spring Boot Test + MockMvc)

- CRUD, asignación y transiciones de tickets
- Seguridad con JWT y restricciones por propietario o técnico

Testcontainers

- Base de datos en contenedor

- Pruebas reproducibles

---

8) Observabilidad: Actuator + Micrometer

Debe exponerse:

- `/actuator/health`
- `/actuator/info`
- `/actuator/metrics`
- `/actuator/prometheus` (si lo deseas)

Métricas clave:

- tiempo medio de primera respuesta y resolución
- tickets vencidos según SLA
- tickets creados por categoría y prioridad
- fallas de autenticación y accesos denegados

---

9) Contenerización y Microservicios

Dockerfile multi-stage

- Etapa build: Maven + JDK
- Etapa run: JRE slim

docker-compose

- support-ticket-service
- technician-dispatch-service
- db
- (opcional) gateway o config-server

Criterios de Aceptación

✓ Funcional

- Registro y autenticación con JWT
- Tickets creados, asignados y gestionados correctamente

- Integración completa con technician-dispatch-service

✓ Arquitectura

- Hexagonal pura, con puertos y adaptadores
- Dominio sin dependencias de frameworks
- MapStruct funcionando

✓ Persistencia

- Relaciones JPA de tickets, usuarios, técnicos e historiales correctas
- Asignaciones y cambios de estado transaccionales

✓ Seguridad

- JWT válido
- Roles de usuario, técnico y administrador aplicados
- Propiedad de tickets y accesos restringidos

✓ Calidad

- Pruebas unitarias + integración
- Testcontainers funcionando

✓ Observabilidad

- Actuator expone métricas
- Logging estructurado

✓ Despliegue

- Dockerfile funcional
- docker-compose operativo

✓ Documentación

- README completo
- Diagramas: arquitectura, ciclo de vida del ticket y relación entre microservicios

---

Entregables

✓ Repositorio GitHub público
✓ Proyecto comprimido (.zip)
✓ Colección de pruebas (Postman u otra) + SWAGGER
✓ README con:

- Descripción del sistema
- Endpoints
- Instrucciones de ejecución local + docker-compose
- Roles, flujo de asignación, transiciones y reglas de SLA
- Capturas de pruebas
  ✓ Diagrama de arquitectura hexagonal
  ✓ Evidencia de métricas, tickets vencidos y logs